

Fabulari

3813ICT - Phase 1 submission

Student	Tariro Kandeya
Student number	s5373405
Workshop	GC Thursday @1100
Repository	https://github.com/TarieBeverly/fabulari

Repository

[Open the private GitHub repository](#)

The teaching staff member must have collaborator access before marking. This document contains an exact copy of the raw Phase1.md source on the following pages. Detailed requirements, architecture, data, API, wireframes and storyboard are linked from that source and included in the repository.

Prototype scope

Working user, group and room management; session authentication; request approval; scoped permissions; age checks; JSON persistence; profiles and audit logs. Chat is a labelled local Phase 1 mock. MongoDB and real-time Socket.IO communication are Phase 2 work.

Validation

Eight backend workflow checks passed. Angular TypeScript/template compilation and the production build passed. Final expanded-interface visual checking remains to be confirmed. The stylesheet size warning was non-fatal.

Raw Phase1.md

Fabulari – Phase 1

- **Name:** Tariro Kandeya
- **Student number:** s5373405
- **Workshop time:** GC Thursday @1100
- **Repository:** <https://github.com/TarieBeverly/fabulari>

1. Project Overview

Fabulari is a community chat application where users join groups and access chat rooms. The Phase 1 prototype implements user, group and room management using Angular 20, Node.js and Express, with server-side JSON persistence. It provides interfaces for Users, Group Admins and Super Admins. Chat is a clearly labelled local mock interface.

2. Requirements and Assumptions

The complete elicitation table and source precedence are in [Requirements](docs/requirements.md). The July 22 and July 29 client briefings supplement the assignment PDF. Later teaching-team clarifications supplied by the student take precedence where they explicitly amend the briefings.

Creation of a group requires a User request approved by Super Admin. The requester becomes the Group Admin. Joining requires a request approved by a Group Admin. Group deletion requires a Group Admin request approved by Super Admin. Deleting another system user requires a Group Admin request approved by Super Admin.

Date of birth is used to calculate age. Minimum ages are checked when joining a group and opening a room. A sole Group Admin must appoint a successor before stepping down or being deleted. Super Admin accounts do not chat and cannot delete themselves.

3. Git Strategy

The private repository uses `main` for milestones and `feature/phase1-prototype` for implementation. Meaningful changes are committed with descriptive messages and pushed regularly. Feature work is checked before merging into `main`.

Designs were committed in `90eee6f`, linked from this report in `62fe729`, and pushed before implementation. Planning was committed in `3dbdfc0`. Backend authentication was committed in `263cbd0`; the initial Angular interface in `ba6bc52`. These are existing history entries, not fabricated commits.

Teaching staff collaborator access must be added before marking; it has not been verified from this local prototype.

4. Data Structures

[Data Structures](docs/data-structures.md) describes users, groups, channels, requests and audit logs. Unique IDs join the records. Passwords are stored as salted bcrypt hashes and excluded from every frontend response. Group administrator IDs define which groups a user can manage. JSON writes use a temporary file followed by rename.

5. Angular Architecture

[Angular Architecture](docs/angular-architecture.md) describes the implemented components, services, models and routes. Login handles sign-in, registration and first-run setup. Dashboard contains the community browser, group administration, request queues, profile, user directory and logs. The consolidated dashboard keeps the Phase 1 prototype small; separate administrative components can be introduced in Phase 2.

AuthService manages authentication, ApiService communicates with the management API, and an authentication guard checks `/auth/me` before opening `/dashboard`. Server checks remain authoritative.

6. Server Endpoints

[API Endpoints](docs/api-endpoints.md) documents methods, routes and permissions. Authentication routes are isolated in `server/auth.js`; management routes are in `server/api.js`. Domain validation and permission helpers are in `server/domain.js`. Successful mutations are written to JSON and logged.

7. Design Documents

- [Wireframes](docs/design/wireframes.md)
- [Storyboard](docs/design/storyboard.md)

These documents were committed and pushed before coding. The original wireframes are retained as design evidence; additions describe the consolidated implemented prototype. The interface uses the supplied logo and its navy, blue, yellow and coral palette. Responsive layouts stack panels on small screens. Controls have labels, keyboard focus styles, visible status/error messages and deletion confirmations.

8. Prototype Scope and Verification

Working functions include registration, login/logout, users, group requests/approval, membership assignment, Group Admin controls, rooms, requests, profiles and JSON persistence. Sample data includes Photography with General and Events rooms, labelled as a demo fixture.

Chat messages/images and participant presence are mock Phase 1 functions. Local chat messages disappear when leaving or refreshing. They do not communicate with other browsers. MongoDB, Socket.IO and real chat history belong to Phase 2.

[Verification](docs/verification.md) records automated backend workflow checks, Angular template compilation, successful production build and remaining browser checks. The local server uses HTTP and in-memory sessions for development.